SOFI-core — Guia de Apresentação para Desenvolvedores

> **Documento de onboarding técnico** para apresentação do projeto SOFI a outros desenvolvedores.
> Use este guia para explicar o que é o sistema, como ele funciona por dentro e como dar manutenção.

---

1. Resumo Executivo — O que é o SOFI?

O **SOFI (Sistema Operacional Financeiro Integrado)** é um hub corporativo de automação financeira desenvolvido sob medida para uma franqueada do **Grupo Boticário** que administra **43 lojas** nas regiões de **Uberlândia (UDI)** e **Belo Horizonte (BH)**.

O problema que ele resolve:

O setor financeiro gastava **2 a 3 horas diárias** em tarefas manuais e repetitivas: acessar portais, baixar relatórios, copiar arquivos, fazer upload, dar baixa em pagamentos e conciliar vendas. Tudo isso multiplicado por 43 lojas.

A solução:

O SOFI automatiza **+80%** dessas atividades, reduzindo o tempo operacional diário para **~5 minutos**. É um servidor Windows dedicado que roda 24/7 como serviço do Windows, com agendamentos automáticos pela manhã, notificações por e-mail e um dashboard web para monitoramento.

Impacto real:

Métrica	Antes do SOFI	Depois do SOFI
Tempo operacional diário	2h — 3h	~5 minutos
Processamento de emails VD	Manual, +30 min	2 minutos (automático)
Extração de relatórios (42 lojas)	Manual, ~1h	Automático, ~15 min total
Conciliação de vendas	Manual, drag-and-drop humano	RPA automatizado
Risco de erro humano	Alto	Virtualmente zero

---

2. Stack Tecnológica

Tecnologia	Versão Mínima	Uso no SOFI
Python	3.10+ (recomendado 3.11)	Linguagem base de todo o projeto

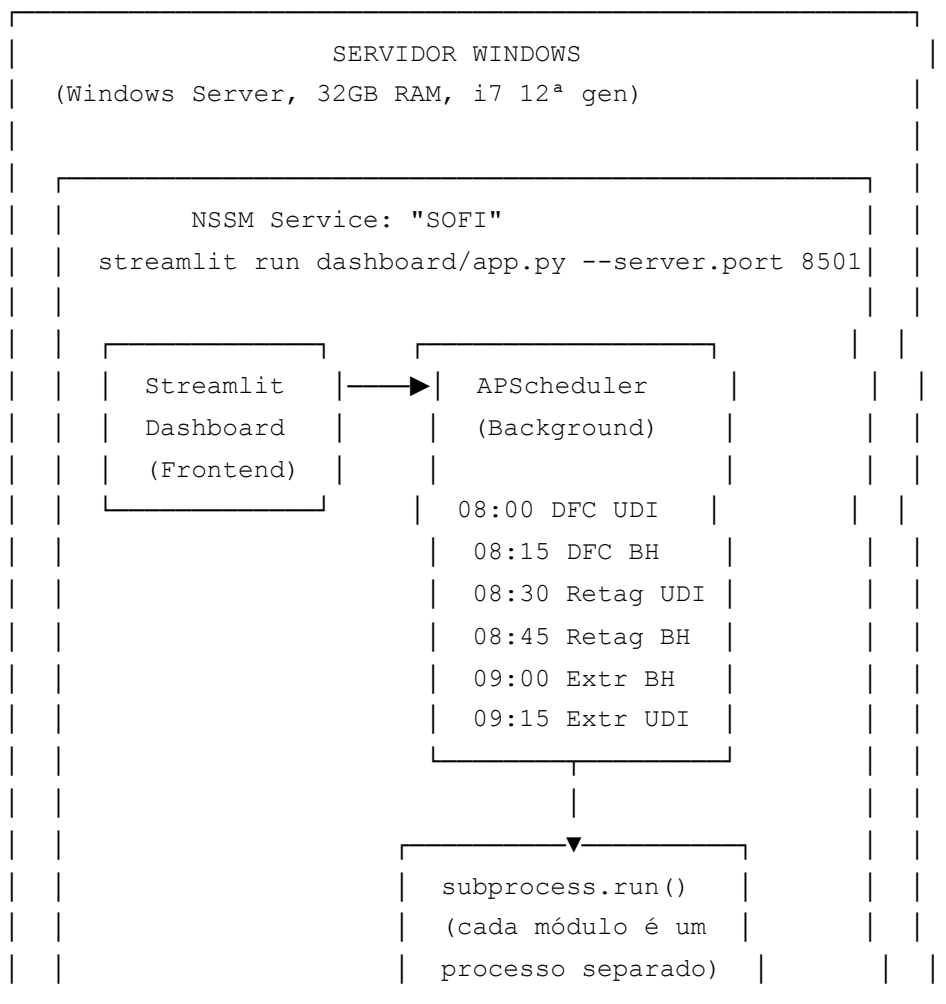
Tecnologia	Versão Mínima	Uso no SOFI
Streamlit	1.58+	Dashboard web de monitoramento e controle
Playwright	1.60+	Automação de navegador (portais FinançasGB, Extranet, Retaguarda)
Selenium	4.0+	Automação de drag-and-drop (Conciliação de vendas)
APScheduler	3.11+	Orquestração de jobs em background (cron interno)
SQLite	Nativo	Banco de logs e auditoria (modo WAL)
pandas	2.0+	Manipulação de dados tabulares
openpyxl / xlrd / xlwt	—	Leitura e escrita de planilhas Excel
pywin32	306+	Integração COM com Outlook e Excel
lxml	4.9+	Parser de XMLs de notas fiscais
python-dotenv	1.0+	Gerenciamento de variáveis de ambiente
smtplib	Nativo	Notificações por e-mail via Gmail SMTP

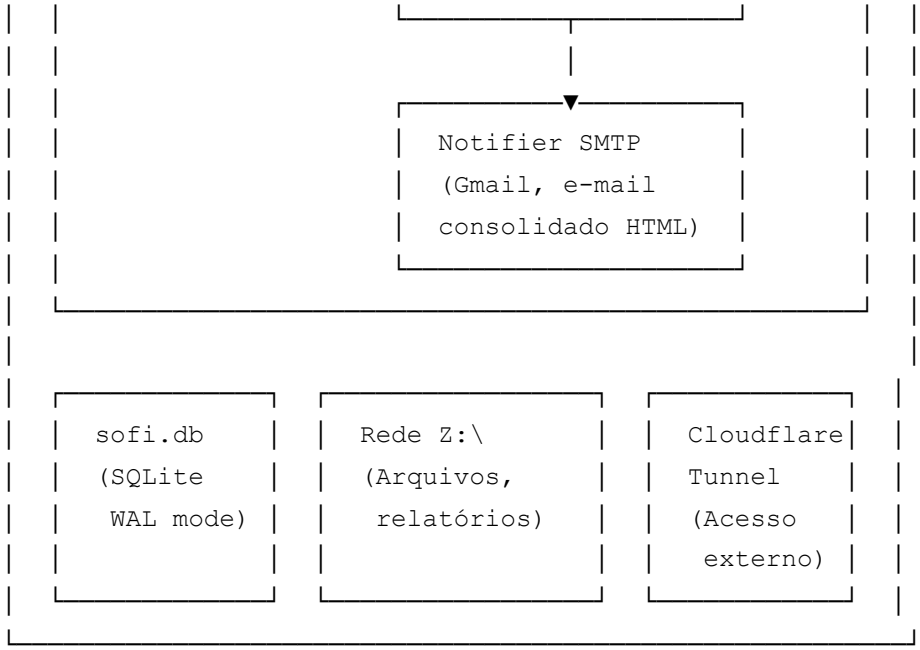
Por que duas ferramentas de automação de navegador?

- **Playwright** é usado na maioria dos módulos (Extranet, Retaguarda, DFC, Importar FinançasGB) porque é mais rápido, mais estável e tem melhor suporte para downloads e uploads.
- **Selenium** é usado exclusivamente no módulo de **Conciliação de Vendas** porque o `ActionChains.drag_and_drop()` do Selenium funciona melhor para arrastar elementos visuais no portal FinançasGB (um portal baseado em Knockout.js que exige interação de DOM nativa).

---

3. Arquitetura — Visão de Alto Nível





Princípios de design:

1. **Desacoplamento:** O dashboard é apenas um visualizador/disparador. A lógica de negócio roda em scripts independentes via subprocess.
2. **Isolamento:** Cada departamento (`financeiro/` , `rh/` , `compras/`) é uma unidade isolada com suas próprias variáveis de ambiente (`.env.local`).
3. **Observabilidade:** Todo script registra logs no banco SQLite via decorator `@log_execution()` .
4. **Resiliência:** O scheduler tem retry automático com lógica de exit codes (0=sucesso, 1=retry, 2=falha permanente).

---

4. Estrutura de Diretórios — O que fica onde?

```
sofi-core/
├── core/              → Infraestrutura compartilhada (NÃO contém lógica de negócio)
│   ├── database/      → SQLite, logger decorator
│   ├── notifier/      → Envio de e-mails HTML consolidados
│   ├── scheduler/     → APScheduler + configs JSON
│   └── sofi_env.py    → Carregamento de .env global + .env.local
│
├── dashboard/         → Interface web (Streamlit)
│   ├── app.py         → Entry point (inicializa scheduler como singleton)
│   ├── style.css      → CSS customizado (padrão visual Sage)
│   ├── components/    → Componentes reutilizáveis (topbar, footer, badge)
│   └── pages/         → Páginas do Streamlit (1 por módulo)
│
├── departments/       → Lógica de negócio organizada por departamento
│   ├── financeiro/
│   │   ├── .env.local → Variáveis exclusivas do financeiro
│   │   └── modules/   → Scripts de automação
│   ├── rh/            → (placeholder)
│   └── compras/       → (placeholder)
```

```
│
├── docs/                → Documentação legada
├── MDS/                 → Planos de ação e conceitos
├── .env                 → Credenciais globais (NÃO versionado)
└── .env.example         → Template de credenciais
```

Regra de ouro:

> **Código de infraestrutura** (logger, scheduler, env loader) → `core/`
> **Código de negócio** (robôs, automações, RPAs) → `departments/<dept>/modules/`
> **Um departamento NUNCA importa de outro departamento.**

---

5. Os 11 Módulos do Financeiro — Explicação Detalhada

5.1 Fluxo Analítico DFC (`fluxo_caixa/`)

- **O que faz:** Acessa o portal FinançasGB via Playwright, navega até o menu "Fluxo de Caixa" → aba "Fluxo Analítico - DFC", preenche os filtros de data (mês atual até mês atual + 12 meses), aguarda o relatório gerar dinamicamente e baixa o Excel.
- **Onde salva:** `Z:\FLUXO ANALÍTICO DIÁRIO\<UDI|BH>\<ANO>\<MÊS>\<DIA>\`
- **Particularidade:** Usa `dateutil.relativedelta` para calcular o período de 12 meses à frente. Tem lógica de fallback com múltiplos seletores CSS/XPath para lidar com mudanças no portal.

5.2 Retaguarda (`retaguarda/`)

- **O que faz:** Extrai relatórios de faturamento diário do portal Retaguarda e faz upload automático no FinançasGB.
- **Unificado:** Um único script `extrator_retaguarda.py` serve UDI e BH via parâmetro `--unidade`.
- **Disparo manual:** Possui `executar_relatorio.bat` para execução independente do scheduler.

5.3 Extranet (`extranet/`)

- **O que faz:** Acessa a Extranet Boticário via Playwright, baixa CSVs de vendas diárias D-1, e automaticamente chama `importar_financas.py` para upload no FinançasGB.
- **Separado por região:** Scripts independentes em `BH/` e `UDI/`.

5.4 Monitor de Emails VD (`AUTOMAÇÃO EMAILS VD/`)

- **O que faz:** Monitora em background as caixas de e-mail de Venda Direta no Outlook via COM (pywin32). Ao detectar novos e-mails, extrai os anexos XLSX, converte para XLS (formato compatível com Oracle) e salva na rede.
- **Separado por região:** `monitor_emails_vd.py` (UDI) e `BH/monitor_emails_vd_BH.py`.
- **Inicialização:** Via `iniciar_monitor.bat`.

- **Log próprio:** `monitor_vd.log` para debug independente.

5.5 XML / MeuDanfe (`xml/debug_meudanfe.py`)

- **O que faz:** Lê planilhas com chaves de acesso NFe, envia para a API MeuDanfe v2 e baixa os XMLs da SEFAZ.
- **API Key:** Configurada em `MEUDANFE_API_KEY` no `.env`.

5.6 Conversor XML SEFAZ (`xml/conversor_xml_sefaz.py`)

- **O que faz:** Converte XMLs de NF-e do formato proprietário MeuDanfe para o padrão oficial da SEFAZ (nfeProc v4.00).
- **O que corrige:**
  - Ordem dos blocos (protNFe antes de NFe → NFe antes de protNFe)
  - Atributos como sub-elementos (`Id` , `nItem` , `versao`)
  - Ordem interna de TODOS os elementos recursivamente
  - Atributos `xmlns` e `versao` ausentes no `protNFe`
- **Uso standalone:** `python conversor_xml_sefaz.py "caminho_pasta" --backup`
- **Uso programático:** Chamado pelo `gerador_relatorio.py` se disponível.

5.7 Gerador de Relatórios Cálamo (`xml/gerador_relatorio.py`)

- **O que faz:** Gera 4 relatórios Excel formatados (Geral BH, Geral UDI, Matheus BH, Matheus UDI) a partir dos XMLs processados.
- **Usa Excel COM:** `win32com.client.DispatchEx('Excel.Application')` para manipular planilhas .xls com fórmulas VLOOKUP, SUMIFS, formatação condicional, merge de células e cores alternadas por loja.
- **Features avançadas:**
  - Resumo por canal (Loja / VD / Eudora) com cálculo de % ICMS ST
  - Prazos de pagamento configuráveis por PDV via `regras_prazo.txt`
  - Cabeçalho com semana calculada (segunda a domingo)
  - Backup temporário de blocos de Totais/Resumo em aba auxiliar

5.8 Importar FinançasGB (`importar_financas.py`)

- **O que faz:** Utilitário compartilhado que faz login no portal FinançasGB via Playwright e realiza upload automático de arquivos.
- **Recursos anti-bot:** `STEALTH_INIT_SCRIPT` remove `navigator.webdriver` , injeta plugins/languages realistas.
- **Bypass de modais:** Trata modal de onboarding (botões Avançar/Fechar) e remove overlays/widgets (Maze, Zendesk).
- **Dois modos:** Chamável como função (`upload_para_financas()`) ou modo standalone interativo.

5.9 Conciliação de Vendas (`conciliacao_venda/`)

- **O que faz:** RPA (Robotic Process Automation) que automatiza a conciliação financeira no portal FinançasGB. O robô:
    1. Lê a planilha de fechamento de caixa da rede corporativa
    2. Filtra e agrupa transações por tipo (Omnichannel, complexos)
    3. Abre o Chrome no FinançasGB via Selenium
    4. Localiza "Caixas Laranjas" (vendas PDV) e "Caixas Azuis" (NSU adquirente) por valor/título
    5. Realiza **drag-and-drop físico** via `ActionChains` para conciliar
    6. Preenche justificativas automaticamente quando exigido
- **3 versões:**
    - `robo_conciliacao_omini.py` — Omnichannel (headless/terminal)
    - `robo_conciliacao_omini_gui.py` — Omnichannel com GUI (CustomTkinter verde/preto Boticário)
    - `robo_conciliacao_oficial_complexos.py` — Casos 1-para-N com sistema de checkpoint (retomada)
- **Compilável:** Possui `.spec` para PyInstaller → `.exe` distribuível.

5.10 Depósitos via API (`depositos/`)

- **O que faz:** Automatiza o lançamento de depósitos no FinançasGB via API REST.
- **Fluxo:** Captura o token de sessão via Playwright (`capturar_token.py`), depois faz chamadas REST com `requests` para a API `FechamentoDeCaixaAPI/RealizarDeposito/`.
- **Status:** Em evolução. Possui planos de expansão para todas as lojas.

5.11 Leitor XML (`xml/leitor_xml.py`, `read_xls.py`)

- **O que faz:** Parser de XMLs de NF-e que extrai dados estruturados (PDV, número do documento, data de emissão, vencimento, totais).
- **Apoio:** `read_xls.py` para leitura de planilhas .xls legadas do SAP/Oracle.

---

6. Sistema de Logging e Observabilidade

Como funciona o `@log_execution()`:

```
from core.database.logger import log_execution

@log_execution(modulo="Meu Módulo")
def main():
    print("Isso vai pro terminal E pro banco de dados")
    # Qualquer print() ou traceback é capturado automaticamente
```

Fluxo interno:

1. `INSERT INTO execution_logs` com status `INICIADO`

2. `sys.stdout` e `sys.stderr` são substituídos por `DBStreamCatcher`
3. Cada `print()` faz append na coluna `mensagem` do registro no SQLite
4. No `finally`, status é atualizado para `SUCESSO` ou `ERRO`

Banco SQLite (`sofi.db`):

- Modo **WAL** (Write-Ahead Logging) para leituras simultâneas sem bloqueio
- Timeout de 10s por conexão
- Caminho configurável via `LOG_DB_PATH` no `.env`

---

7. Sistema de Agendamento (Scheduler)

Como funciona:

1. O `app.py` do Streamlit inicializa o scheduler como **singleton** via `@st.cache_resource`
2. O scheduler lê `scheduler_config.json` e agrupa jobs por horário
3. Para cada grupo, agenda um `APScheduler.add_job()` do tipo cron
4. No horário configurado, `run_all_jobs()` executa cada job sequencialmente via `subprocess.run()`
5. Cada job tem retry automático baseado no exit code retornado
6. Ao final de todos os jobs, o Notifier envia e-mail consolidado

Configuração (`scheduler_config.json`):

```
{
  "enabled": true,
  "timezone": "America/Sao_Paulo",
  "notification": {
    "enabled": true,
    "send_on_success": true,
    "send_on_error": true
  },
  "jobs": [
    {
      "name": "Fluxo Analítico DFC UDI",
      "enabled": true,
      "script": "departments/financeiro/modules/fluxo_caixa/extrator_dfc.py",
      "schedule": { "type": "cron", "hour": 8, "minute": 0 },
      "retry": { "max_attempts": 3, "delay_seconds": 120 }
    }
  ]
}
```

Para adicionar um novo job:

1. Adicione a entrada no `scheduler_config.json`
2. Certifique-se de que o script usa `sys.exit(0/1/2)` corretamente
3. O scheduler será recarregado na próxima reinicialização do serviço

---

8. Gerenciamento de Variáveis de Ambiente

Hierarquia de carregamento:

```
1. .env (raiz)              → Credenciais globais (todos os departamentos)
2. departments/<dept>/.env.local → Credenciais exclusivas do departamento (sobrescreve)
```

Como usar em qualquer script:

```
import sys
from pathlib import Path

# Resolve a raiz do projeto
_ROOT = Path(__file__).resolve()
while not (_ROOT / "core").exists() and _ROOT != _ROOT.parent:
    _ROOT = _ROOT.parent
sys.path.insert(0, str(_ROOT))

from core.sofi_env import load_sofi_env
load_sofi_env(department="financeiro")  # carrega .env + .env.local do dept

import os
email = os.getenv("FINANCAS_UDI_USER")
```

Variáveis críticas do `.env`:

Variável	Obrigatória	Descrição
`FINANCAS_UDI_USER` / `FINANCAS_UDI_PASS`		Login FinançasGB (UDI)
`FINANCAS_BH_USER` / `FINANCAS_BH_PASS`		Login FinançasGB (BH)
`RETAGUARDA_USER` / `RETAGUARDA_PASS`		Login Retaguarda
`EXTRANET_USER` / `EXTRANET_PASSWORD`		Login Extranet
`MEUDANFE_API_KEY`		API de download de XMLs
`HEADLESS`		`true` em produção, `false` para debug
`BROWSER_CHANNEL`		`chrome` ou `msedge`
`DISABLE_HTTP2`		`true` recomendado
`SMTP_USER` / `SMTP_PASSWORD`		Gmail para notificações
`SCHEDULER_NOTIFY_EMAILS`		Destinatários separados por `;`

---

9. Como Configurar o Ambiente de Desenvolvimento

Passo a passo:

```
# 1. Clone o repositório
git clone https://github.com/[org]/sofi-core.git
cd sofi-core

# 2. Crie e ative um venv (recomendado)
python -m venv .venv
.venv\Scripts\activate

# 3. Instale as dependências
pip install -r requirements.txt

# 4. Instale o navegador para Playwright
playwright install chromium

# 5. Copie e preencha o .env
copy .env.example .env
# Edite .env com as credenciais reais

# 6. Rode o dashboard
streamlit run dashboard/app.py
# Acesse http://localhost:8501

# 7. Para testar um módulo isolado
cd departments/financeiro/modules/fluxo_caixa
python extrator_dfc.py --data 23/07/2026
```

Requisitos do Windows:

- **Python 3.10+** (com "Add to PATH")
- **Google Chrome** ou **Microsoft Edge** instalado
- **Microsoft Excel** instalado (para `gerador_relatorio.py`)
- **Microsoft Outlook** instalado (para `monitor_emails_vd.py`)
- **Acesso à rede** `\\srv-dc01\File Server\` (mapeado como `Z:\`)

---

10. Padrões de Código — Convenções do Projeto

Resolução da raiz do projeto:

Todos os scripts usam o mesmo padrão para encontrar a raiz:

```
_ROOT = Path(__file__).resolve()
while not (_ROOT / "core").exists() and _ROOT != _ROOT.parent:
    _ROOT = _ROOT.parent
sys.path.insert(0, str(_ROOT))
```

Stealth anti-bot (para Playwright):

```
from departments.financeiro.modules.importar_financas import (
    CREDENCIAIS, FINANCAS_URL, REALISTIC_USER_AGENT,
    STEALTH_INIT_SCRIPT, browser_args, _fazer_login,
)
```

Exit codes para o scheduler:

```
EXIT_OK        = 0   # Sucesso
EXIT_RETRYABLE = 1   # Falha transitória (Playwright timeout, etc.)
EXIT_PERMANENT = 2   # Falha permanente (dados vazios, etc.)

if __name__ == "__main__":
    try:
        main()
        sys.exit(EXIT_OK)
    except Exception:
        sys.exit(EXIT_RETRYABLE)
```

Decorator de log:

```
from core.database.logger import log_execution

@log_execution(modulo="Nome do Módulo")
def main():
    ...  # todo print() vai pro banco automaticamente
```

---

11. Fluxo de Deploy em Produção

```
1. git pull no servidor Windows
2. pip install -r requirements.txt (se houver mudanças)
3. playwright install chromium (se houver atualização)
4. nssm restart SOFI
5. Verificar: nssm status SOFI → SERVICE_RUNNING
6. Acessar dashboard e verificar badge do scheduler:   Ativo
```

Dica de troubleshooting:

- Se o dashboard não abrir: `nssm restart SOFI`
- Se automações Playwright falharem: trocar `BROWSER_CHANNEL=chrome` por `msedge` no `.env`
- Se Excel COM travar: `taskkill /F /IM EXCEL.EXE` e reexecutar
- Para ver logs: acessar o módulo no dashboard ou consultar diretamente o `sofi.db`

---

12. Portais e Sistemas Externos Integrados

Portal	URL Base	Protocolo	Módulos que usam
FinançasGB	`financasgb.grupoboticario.com.br`	Playwright / API REST	DFC, Retaguarda, Extranet, Importar, Depósitos, Conciliação
Extranet Boticário	`extranet.grupoboticario.com.br`	Playwright	Extranet BH/UDI
Retaguarda	`cp10488.retaguarda.grupoboticario.com.br`	Playwright	Retaguarda BH/UDI
API MeuDanfe v2	`meudanfe.com.br/api/v2`	REST (requests)	XML / Notas Fiscais
Outlook Desktop	Local (COM)	pywin32	Monitor Emails VD
Excel Desktop	Local (COM)	pywin32	Gerador Relatórios Cálamo
SEFAZ	Indireto (via MeuDanfe)	—	Conversor XML
Rede Corporativa	`\\srv-dc01\File Server\`	SMB	Todos os módulos

---

13. Roadmap — Próximos Passos

Módulo	Status	Descrição
SOFI Depósitos	Em evolução	Expandir automação de depósitos para todas as 43 lojas via API
SOFI Conciliação (hub)	Em evolução	Integrar conciliação ao dashboard web
Emails VD (dashboard)	🕐 Em breve	Painel de monitoramento do monitor VD no dashboard
Depósitos (dashboard)	🕐 Em breve	Painel de execução e logs de depósitos no dashboard
RH — Triagem IA	🕐 Em breve	Processamento de currículos com IA
Compras — Mapeador	🕐 Em breve	Consolidação de tabelas de custos

---

14. Glossário

Termo	Significado
PDV	Ponto de Venda (identificador único de cada loja no Grupo Boticário)
VD	Venda Direta (canal de vendas diretas)
DFC	Demonstração de Fluxo de Caixa (relatório financeiro do FinançasGB)
D-1	Dia anterior (referência temporal dos dados extraídos)
Cálamo	Cálamo Distribuidora — fornecedora do Grupo Boticário
NSU	Número Sequencial Único (identificador da transação na adquirente de cartões)
Omnichannel	Vendas que combinam canais online e offline (PDV + e-commerce)
Retaguarda	Portal de backoffice do Grupo Boticário para gestão de lojas
Extranet	Portal de vendas e relatórios corporativos do Grupo Boticário
FinançasGB	Portal central de gestão financeira do Grupo Boticário
NF-e	Nota Fiscal Eletrônica

Termo	Significado
SEFAZ	Secretaria da Fazenda (órgão que define o padrão XML das NF-e)
WAL	Write-Ahead Logging (modo do SQLite que permite leituras e escritas simultâneas)
NSSM	Non-Sucking Service Manager (utilitário para rodar aplicações como serviço Windows)

Documento gerado em 23/07/2026 · SOFI v2.3.0 · Uso Interno